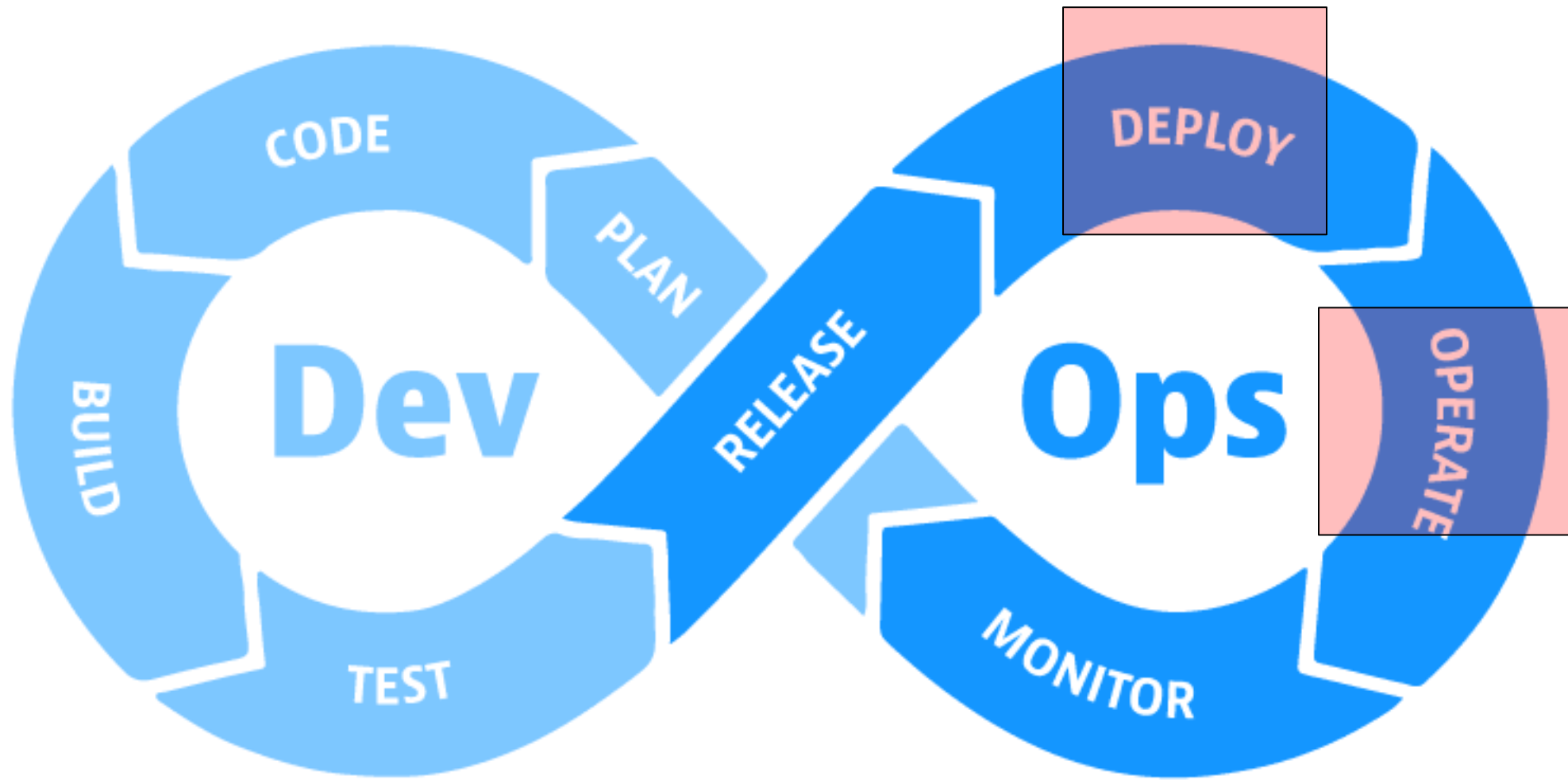


Developer Operations (devops)

Autoscaling and Resources



Build and Release



Think about applications you programmed:
What are symptoms when an application has not enough
resources?

-

Think about applications you programmed: What are the resource-needs for your application?

- At start?
- At normal operation?
- At peak?

How did you determine the needed resources?

How did you “react”?

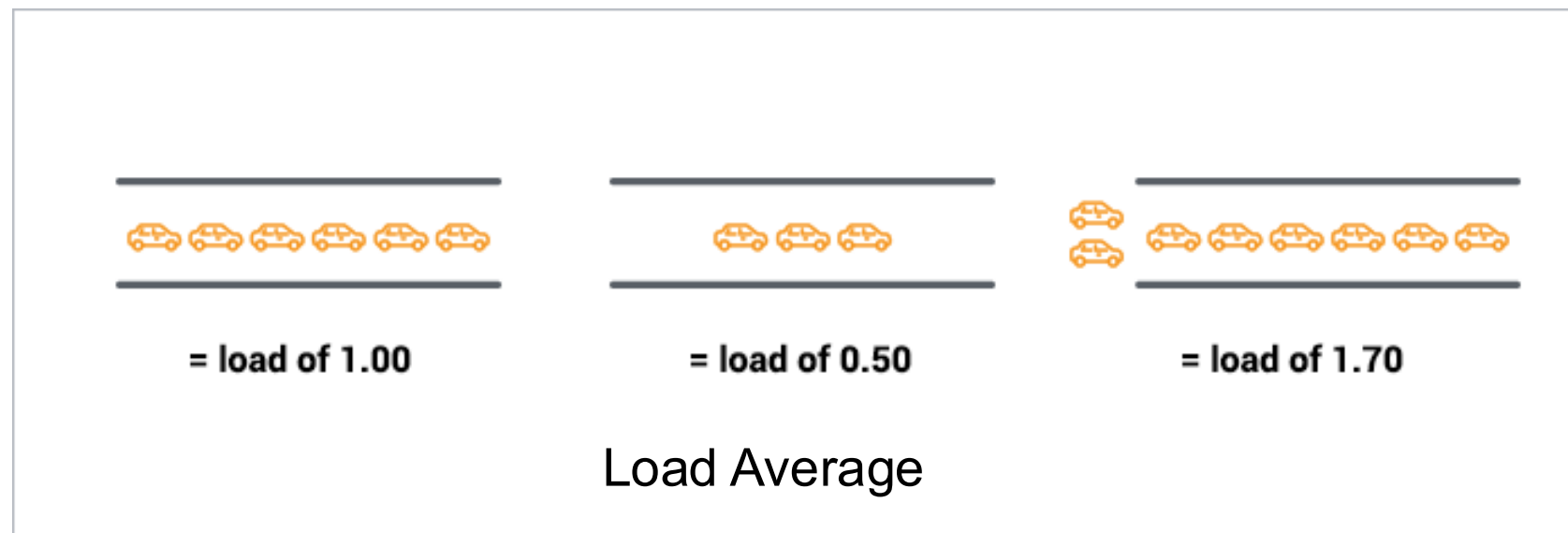
CPU and Memory (and Storage and Network and ...)

CPU

- Load Average goes up
- Context Switching
- System gets slower and slower

Memory

- Swapping is used (if available)
- OutOfMemory Failures generate restarts
- System gets slower and slower



Fail Fast

CPU

- Highly variable, seldomly the main cause
 - Who needs computing anyway?
 - Hard to predict
 - Easy to relax

Memory

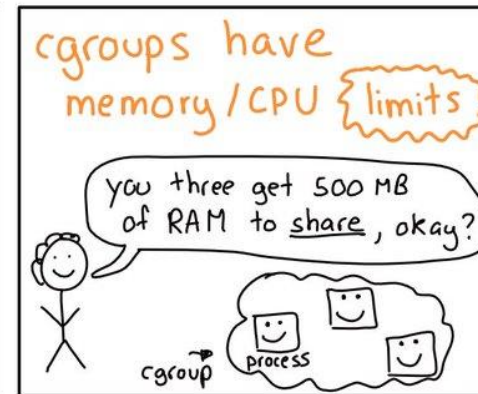
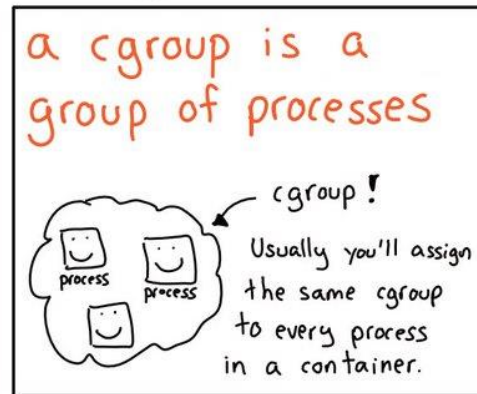
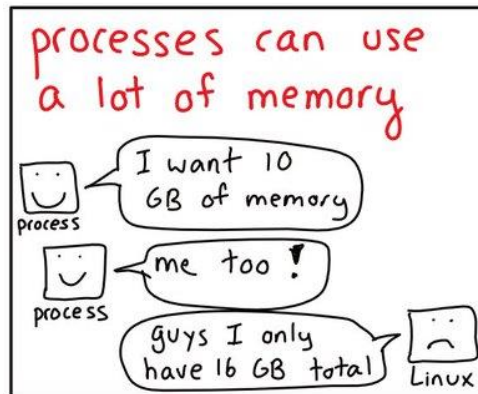
- More static, seldomly the main cause
 - Heapspace anyone?
 - Easier to predict
 - Hard to relax (swap is no solution)

- You need to estimate the consumption of your application:
- Infrastructure is expensive, you are generating the costs
 - Irregular consumption can be a bug (but does not have to)
 - Knowing the limits enables scaling
 - Enabling Fail Fast: If an application behaves weird, kill it

cgroups to the rescue

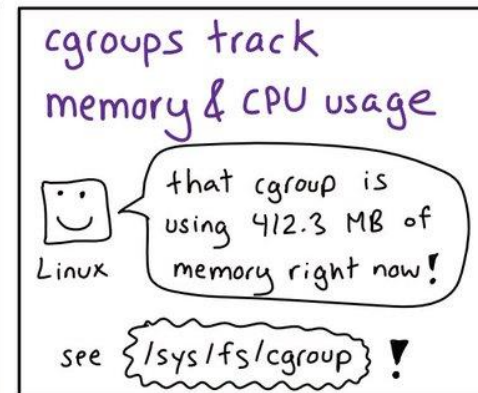
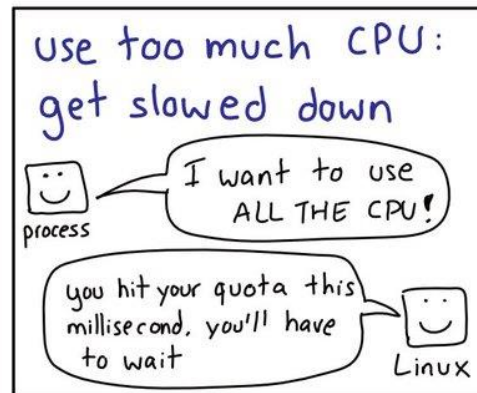
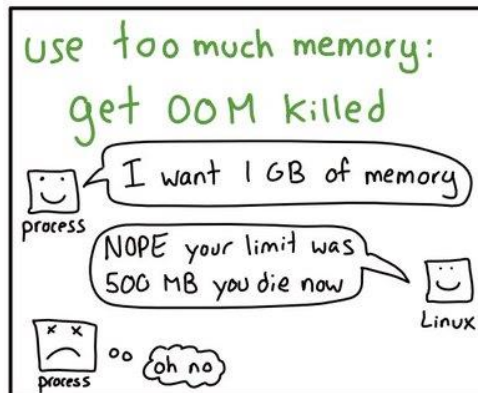
JULIA EVANS
@b0rk

cgroups



cgroup define limits:

- killing process by hitting memory limit
- slowing processes by hitting CPU limit
- tracking usage
- grouping processes



CPU and Memory in Kubernetes

CPU

Measured in CPU-Cores / Virtual Cores

- Might be separated into Millicores
(1/1000 cores)
- Divided into shares
(1 Core = 1000m = 1024 shares)
- Might be the base for billing
- Restricting is not killing the process

Memory

Measured in Megabytes

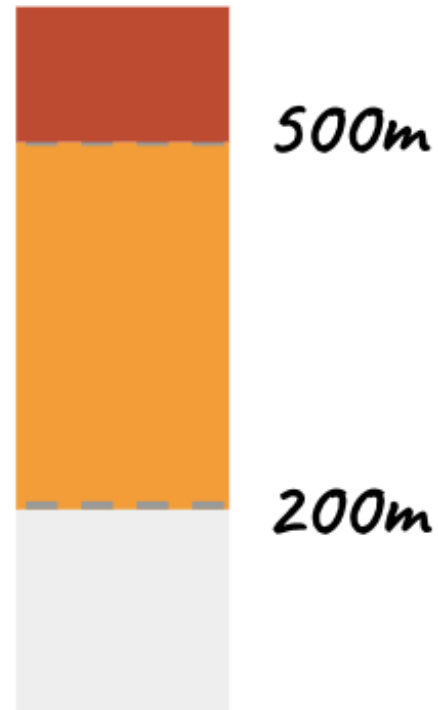
- Might be the base for billing
- Restricting is killing the process

Resources in Kubernetes

CPU

Limit: where CPU
throttling begins

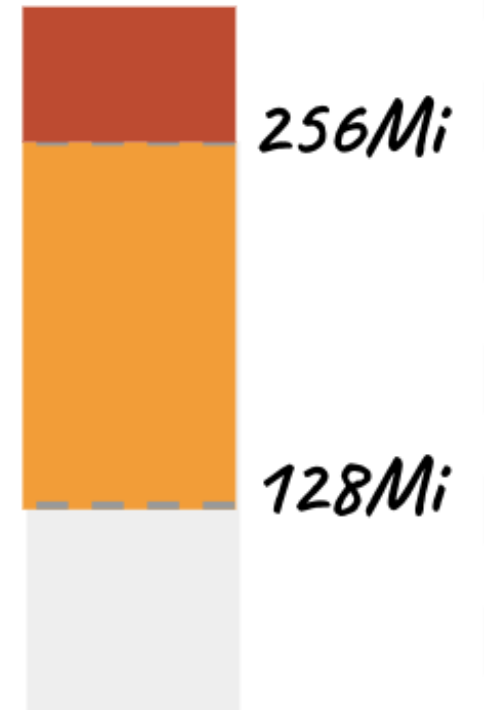
Request: amount
guaranteed by k8s



Memory

Limit: level where
processes are killed

Request: amount
guaranteed by k8s

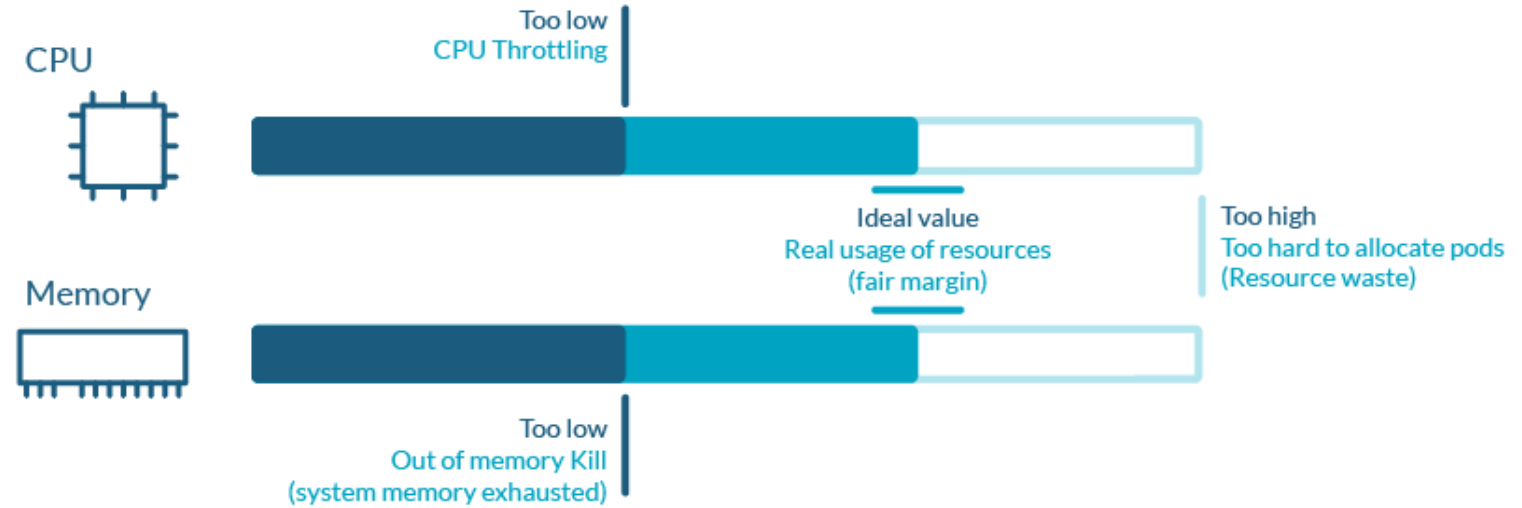


<https://kubernetes.io/docs/concepts/configuration/manage-resources-containers/>

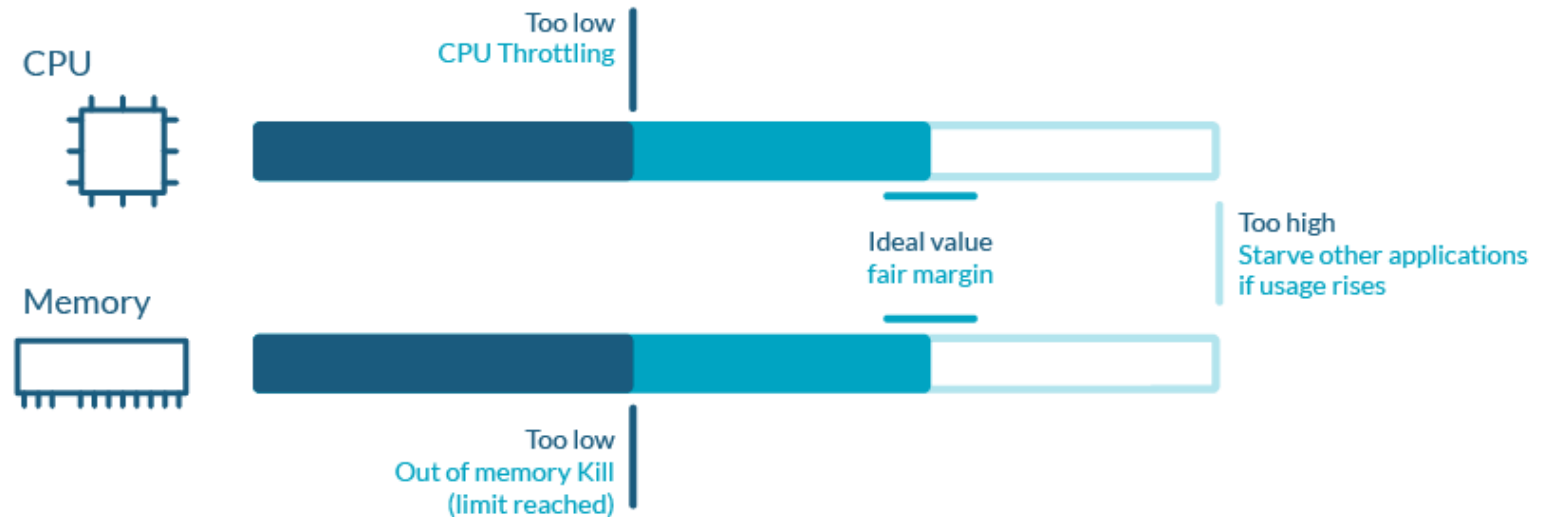
What to define?

Requests → False scheduling
Limits → False App behaviour

Requests



Limits

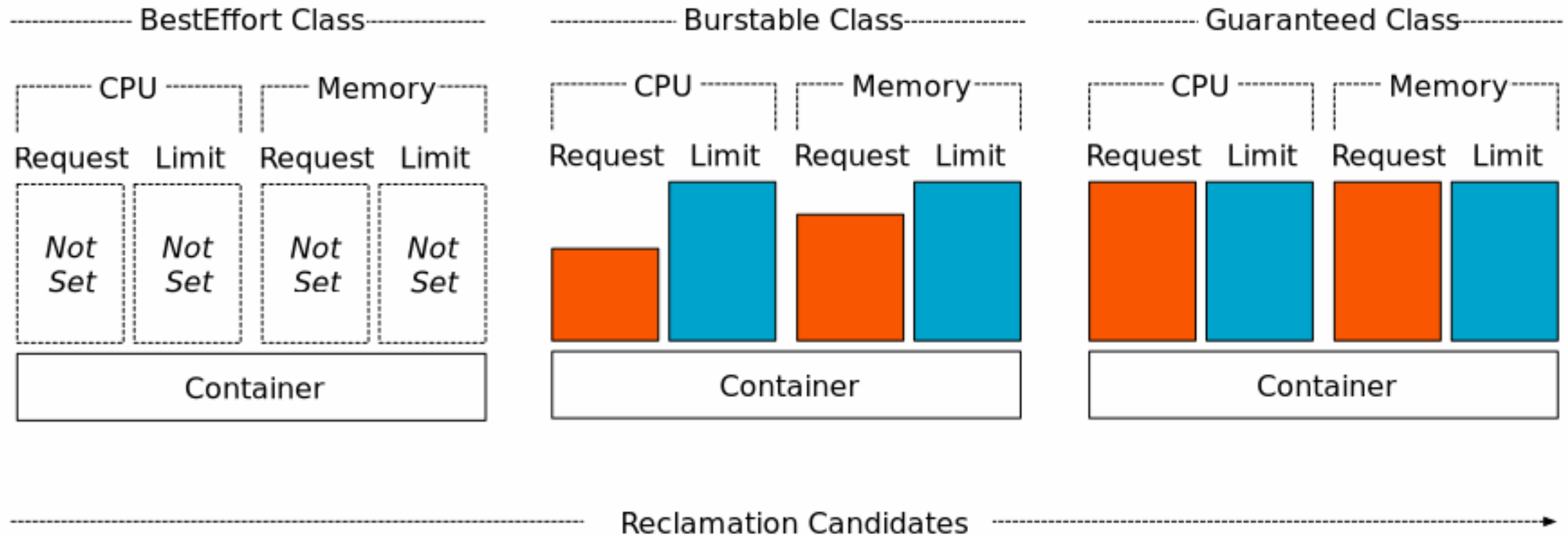


Do you need limits everywhere?

Do you need always memory limits?

...what about CPU limits?

Different Quality of Services



How to find our resources?

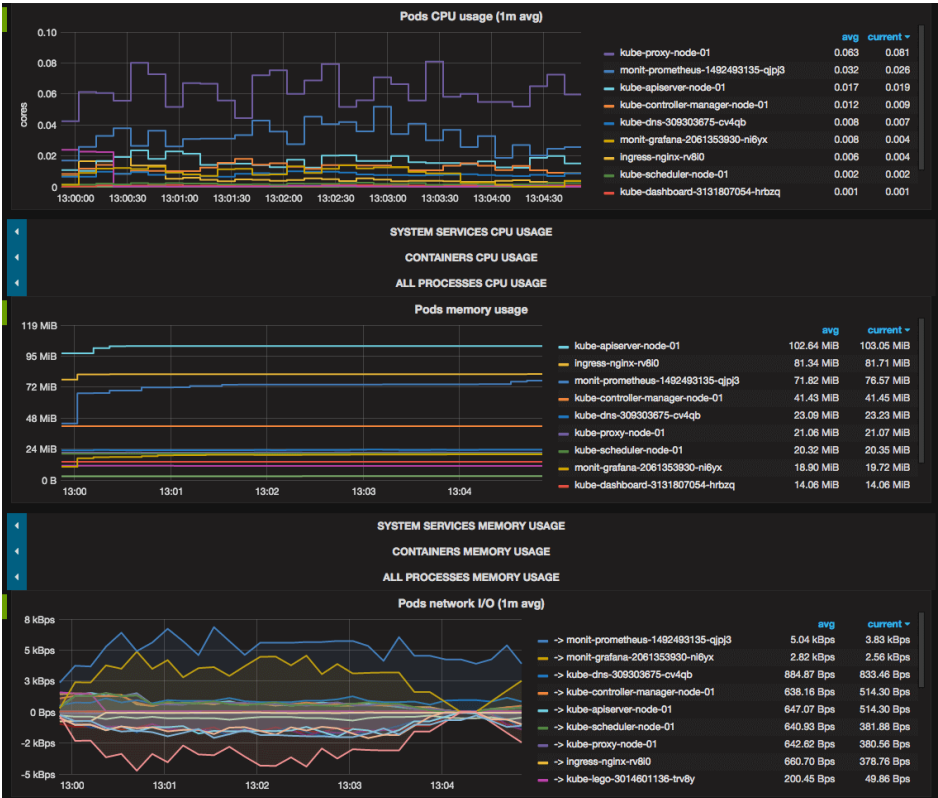


Monitoring Prometheus und Grafana

Metrics Server

```
seg — seg@Gretel — ~ — -zsh — 80x24
[+] ~ kubectl top pods -n chatbots
NAME                                CPU(cores)   MEMORY(bytes)
connecting-worlds-869dc4d6d5-tjtn7  1m           53Mi
eliza-696b57b55f-wnt4r              1m           72Mi
roberta-7d5f9748bb-g6dtr            1m           1095Mi
```

kubectl top pods -n chatbots



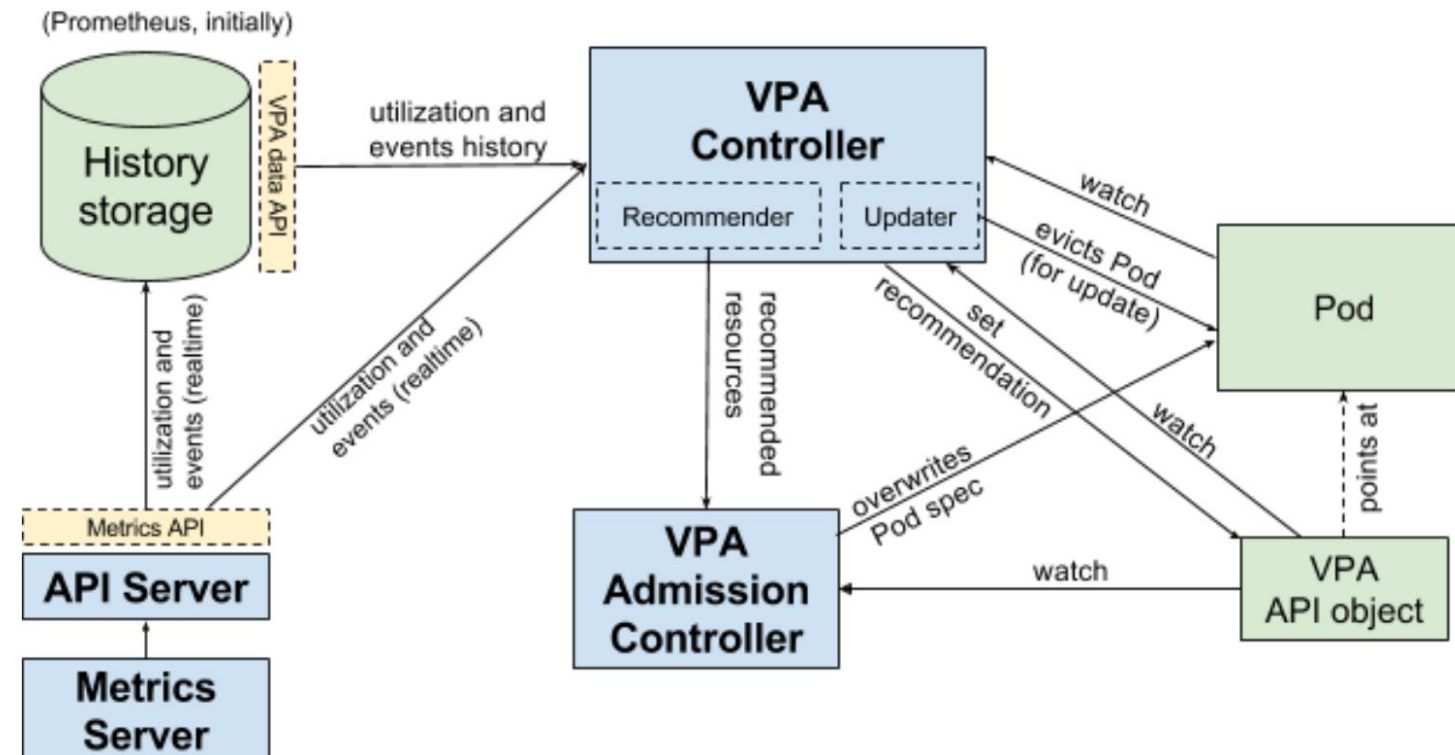
Vertical Pod Autoscaler

Vertical Scaling:

- Increasing/Decreasing existing resources
- Static: needs to adapt existing service (recreation necessary)
- Does not scale infinitely

Kubernetes Installation:

1. Install VPA
2. VPA listens to pod and upon recreation sets resources
3. Values are transparent to be set manually in deployment-config



Horizontal Pod Autoscaler

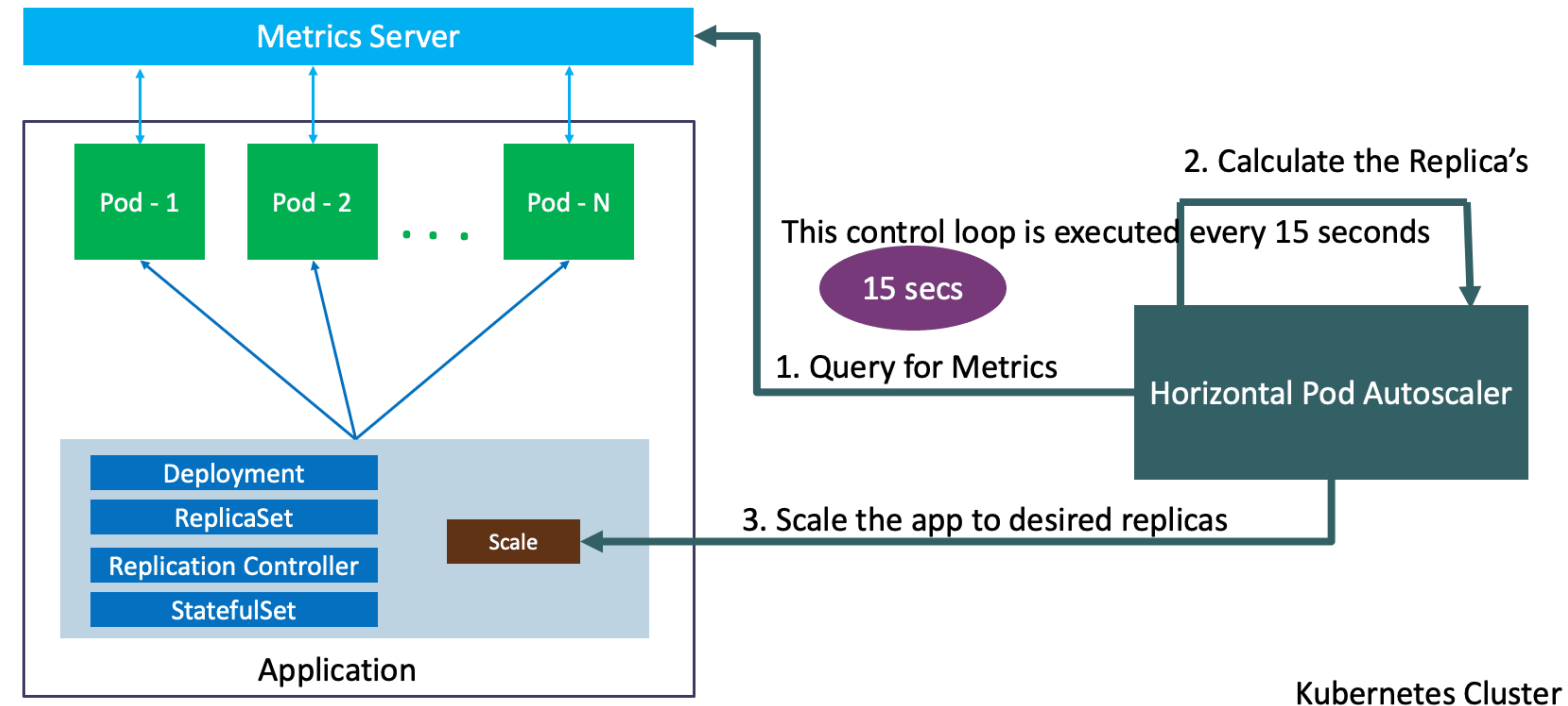
Horizontal Scaling:

- Adding / Removing pods
- Flexible: just creating new resources, not adapting new ones
- Does scale with respect to nodes
- State-Awareness: should not share state

Kubernetes Installation :

```
kubectl autoscale deployment DEPL
--cpu-percent=50 --min=1 --max=10
```

1. HPA listens to pods
2. If metrics are met, add new pods
If metrics are decreasing, remove pods gracefully
3. Can listen to custom metrics



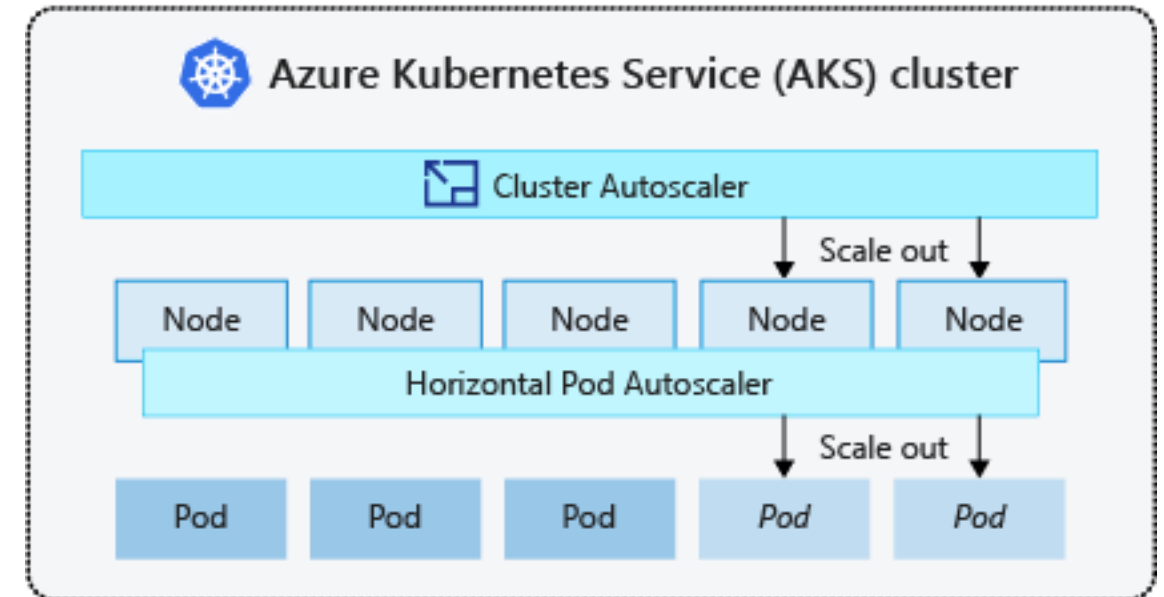
Cluster Autoscaler

Adding and Removing Nodes on demand

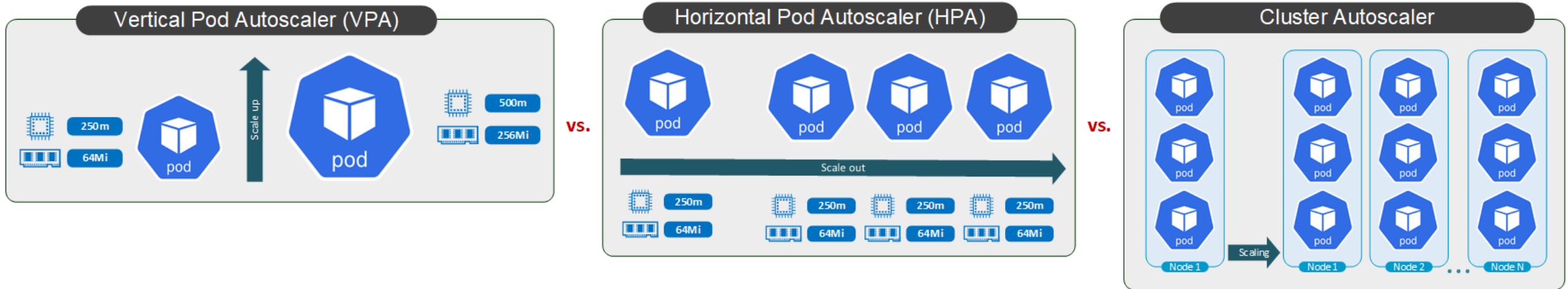
- Adding Removing Nodes
- Needs suitable infrastructure
- Just provisioning worker nodes
 - Needs to be template/pool based

Kubernetes

- Mostly directly configured on infrastructure level
- Can be triggered via ClusterAPI:
<https://cluster-api.sigs.k8s.io/>
- Can work together with HPA:
new Pod → Node full → new Node



Autoscaling Summary



Three Autoscalers: three purposes:

- VPA: defining resources of single pods
- HPA: scaling out
- Cluster: scaling infrastructure

Do not combine VPA / HPA based on CPU

- However, you can use HPA with custom metrics and VPA with CPU

